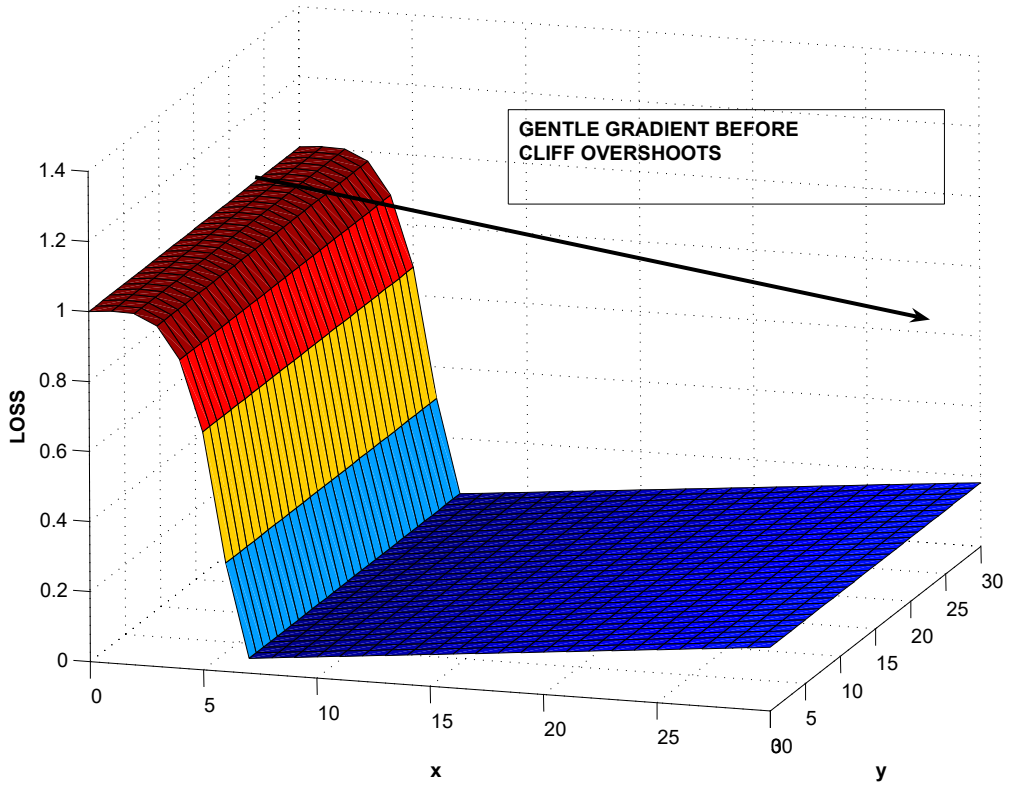
Charu C. Aggarwal
IBM T J Watson Research Center
Yorktown Heights, NY

Second-Order Gradient Descent Methods

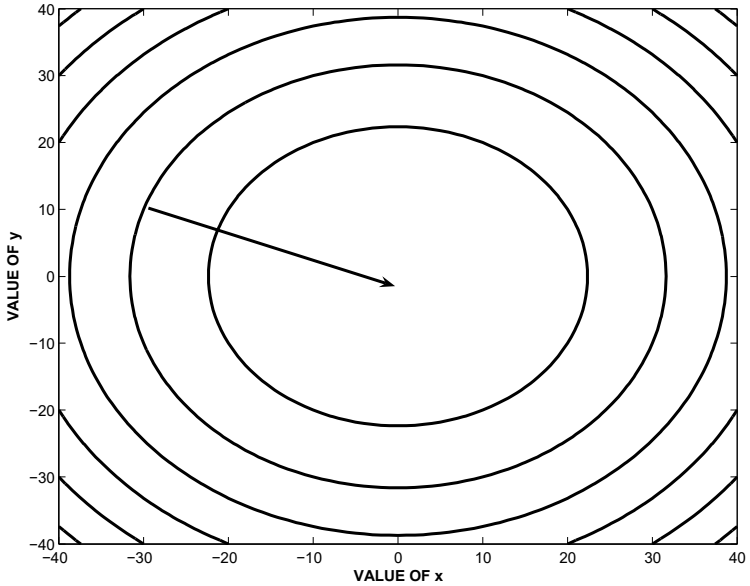
Neural Networks and Deep Learning, Springer, 2018
Chapter 3, Section 3.5.5

Why Second-Order Methods?

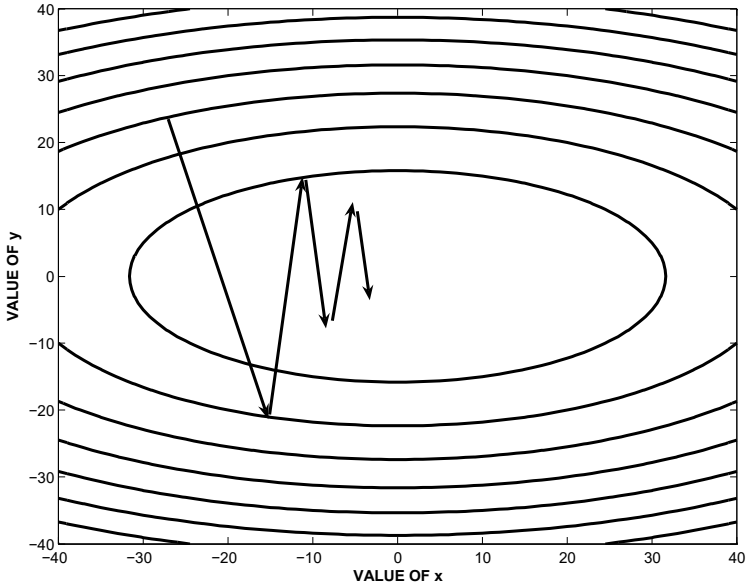


- First-order methods are not enough when there is curvature.

Revisiting the Bowl



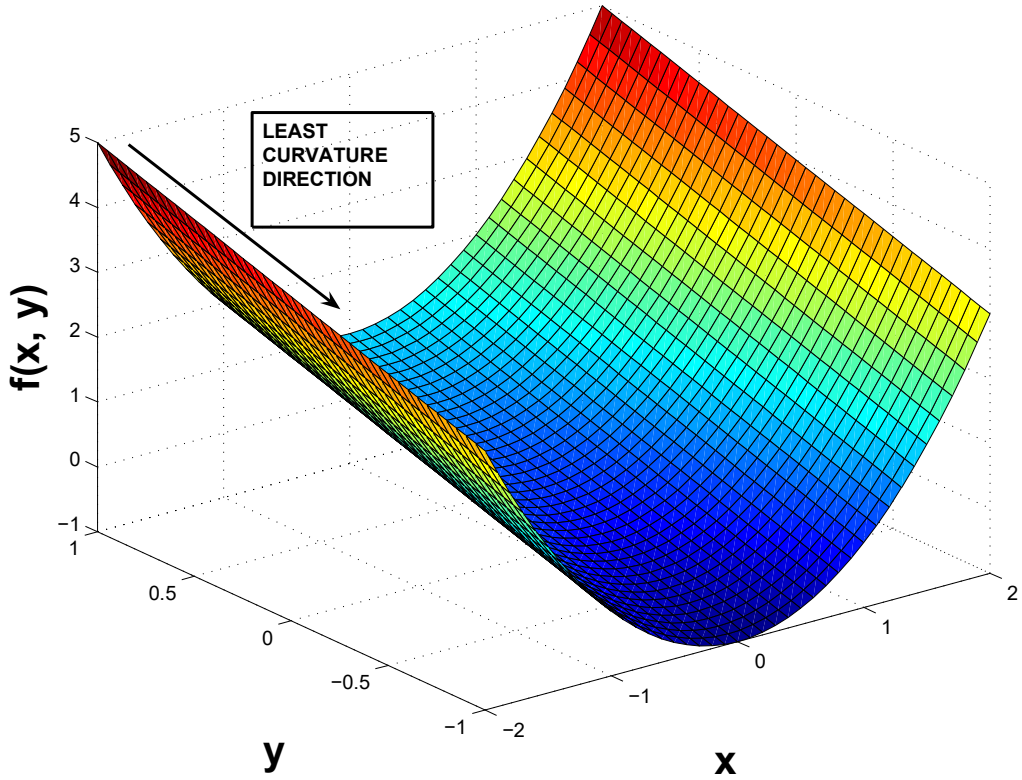
(a) Loss function is circular bowl
$L = x^2 + y^2$



(b) Loss function is elliptical bowl
$L = x^2 + 4y^2$

- High curvature directions cause bouncing in spite of higher gradient $\Rightarrow$ Need second-derivative for more information.

A Valley



- Gently sloping directions are better with less curvature!

The Hessian

- The second-order derivatives of the loss function $L(\overline{W})$ are of the following form:

$$H_{ij} = \frac{\partial^2 L(\overline{W})}{\partial w_i \partial w_j}$$

- The partial derivatives use all pairwise parameters in the denominator.

- For a neural network with d parameters, we have a $d \times d$ *Hessian matrix* H, for which the (i, j)th entry is H_{ij}.

Quadratic Approximation of Loss Function

- One can write a quadratic approximation of the loss function with Taylor expansion about $\overline{W_0}$:

$$L(\overline{W}) \approx L(\overline{W}_0) + (\overline{W} - \overline{W}_0)^T [\nabla L(\overline{W}_0)] + \frac{1}{2}(\overline{W} - \overline{W}_0)^T H (\overline{W} - \overline{W}_0) \tag{19}$$

- One can derive a single-step optimality condition from initial point $\overline{W_0}$ by setting the gradient to 0.

Newton's Update

- Can solve quadratic approximation in one step from initial point $\overline{W_0}$.

  $\nabla L(\overline{W}) = 0$ [Gradient of Loss Function]
  $\nabla L(\overline{W}_0) + H(\overline{W} - \overline{W}_0) = 0$ [Gradient of Taylor approximation]

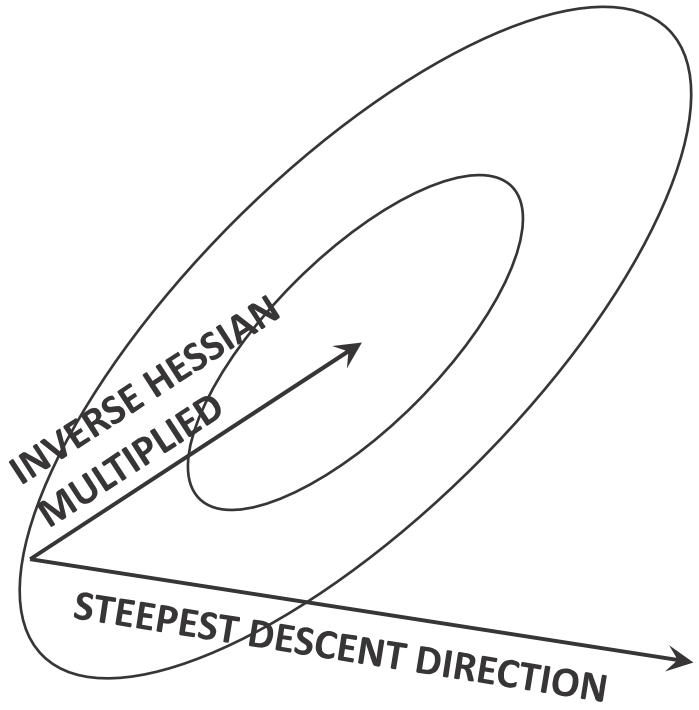
- Rearrange optimality condition to obtain Newton update:

$$\overline{W}^* \Leftarrow \overline{W}_0 - H^{-1}[\nabla L(\overline{W}_0)] \tag{20}$$

- Note the ratio of first-order to second-order $\Rightarrow$ Trade-off between speed and curvature

- Step-size not needed!

Why Second-Order Methods?



- Pre-multiplying with the inverse Hessian finds a trade-off between speed of descent and curvature.

Basic Second-Order Algorithm and Approximations

- Keep making Newton's updates to convergence (single step needed for quadratic function)
  - Even computing the Hessian is difficult!
  - Inverting it is even more difficult

- Solutions:
  - Approximate the Hessian.
  - Find an algorithm that works with projection $H\overline{v}$ for some direction $\overline{v}$.

Conjugate Gradient Method

- Get to optimal in d steps (instead of single Newton step) where d is number of parameters.

- Use optimal step-sizes to get best point along a direction.

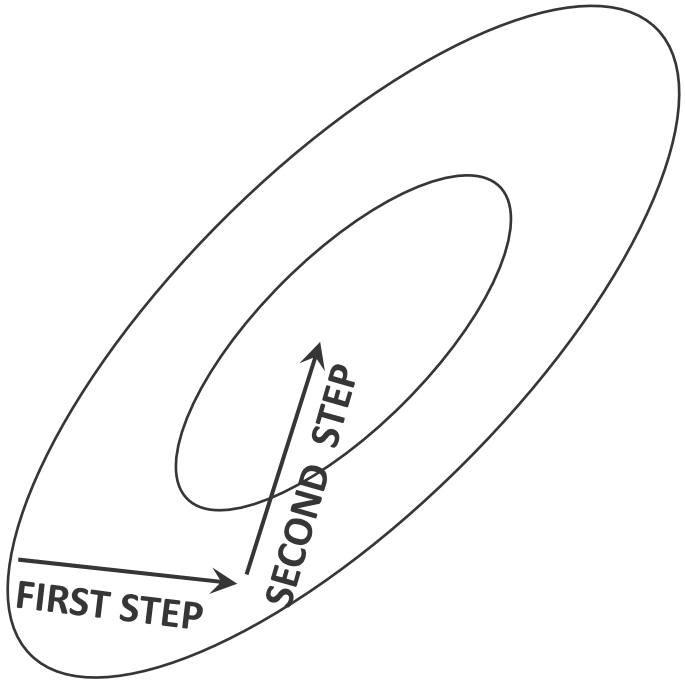
- *Thou shalt not worsen with respect to previous directions!*

- **Conjugate direction:** The gradient of the loss function on *any* point on an update direction is always orthogonal to the previous update directions.

$$\overline{q}_{t+1} = -\nabla L(\overline{W}_{t+1}) + \left(\frac{\overline{q}_t^T H[\nabla L(\overline{W}_{t+1})]}{\overline{q}_t^T H \overline{q}_t} \right) \overline{q}_t \qquad (21)$$

- For quadratic function, it requires d updates instead of single update of Newton method.

Conjugate Gradients on 2-Dimensional Quadratic



- Two conjugate directions are required to reach optimality

Conjugate Gradient Algorithm

- For quadratic functions only.
  - Update $\overline{W}_{t+1} \Leftarrow \overline{W}_t + \alpha_t \overline{q}_t$. Here, the step size α_t is computed using line search.
  - Set $\overline{q}_{t+1} = -\nabla L(\overline{W}_{t+1}) + \left(\frac{\overline{q}_t^T H[\nabla L(\overline{W}_{t+1})]}{\overline{q}_t^T H \overline{q}_t} \right) \overline{q}_t$. Increment t by 1.

- For non-quadratic functions approximate loss function with Taylor expansion and perform $\ll d$ of the above steps. Then repeat.

Efficiently Computing Projection of Hessian

- The update requires computation of the *projection* of the Hessian rather than inversion of Hessian.

$$\overline{q}_{t+1} = -\nabla L(\overline{W}_{t+1}) + \left(\frac{\overline{q}_t^T H[\nabla L(\overline{W}_{t+1})]}{\overline{q}_t^T H \overline{q}_t} \right) \overline{q}_t \tag{22}$$

- Easy to perform numerically!

$$H\overline{v} \approx \frac{\nabla L(\overline{W}_0 + \delta\overline{v}) - \nabla L(\overline{W}_0)}{\delta} \tag{23}$$

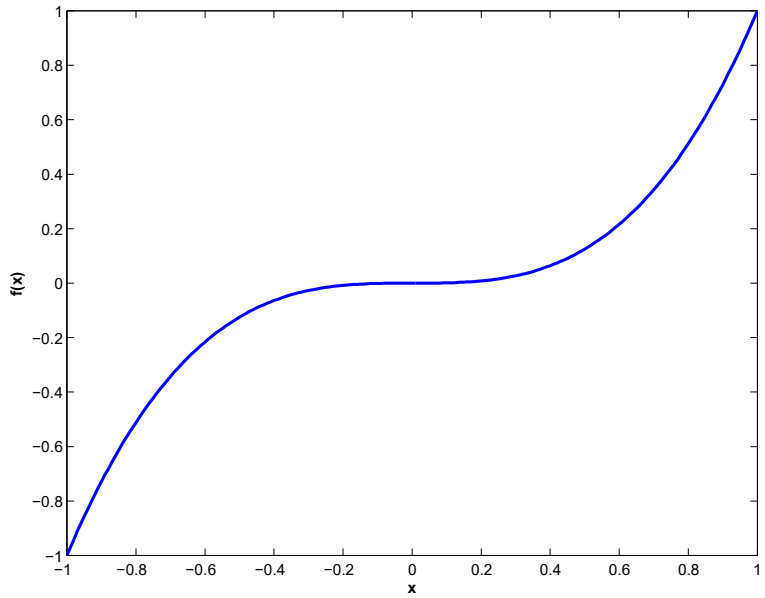
Other Second-Order Methods

- *Quasi-Newton Method:* A sequence of increasingly accurate approximations of the inverse Hessian matrix are used in various steps.

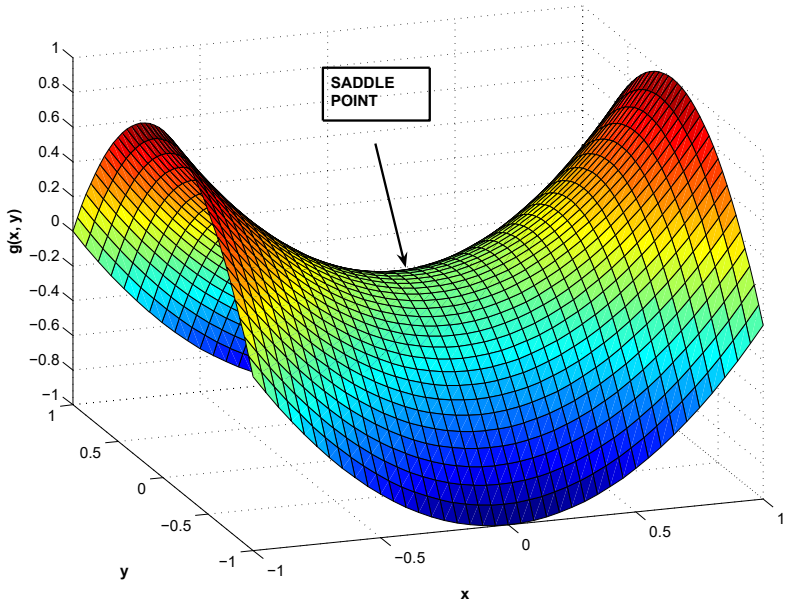
- Many variations of this approach.

- Commonly-used update is BFGS, which stands for the Broyden–Fletcher–Goldfarb–Shanno algorithm and its limited memory variant L-BFGS.

Problems with Second-Order Methods



(a) $f(x) = x^3$
Degenerate



(b) $f(x) = x^2 - y^2$
Stationary

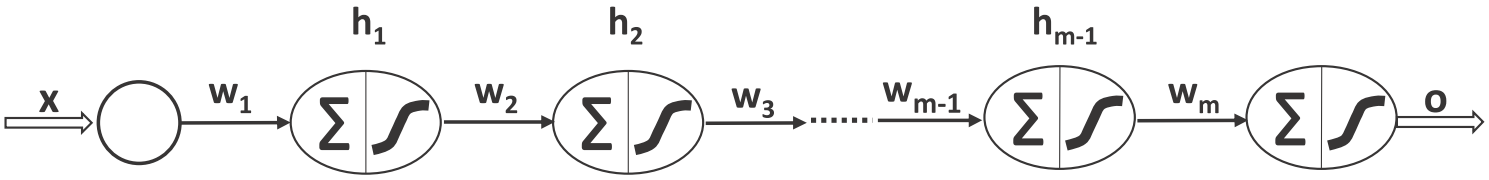
- Saddle points: Whether it is maximum or minimum depends on which direction we approach it from.

Charu C. Aggarwal
IBM T J Watson Research Center
Yorktown Heights, NY

Batch Normalization

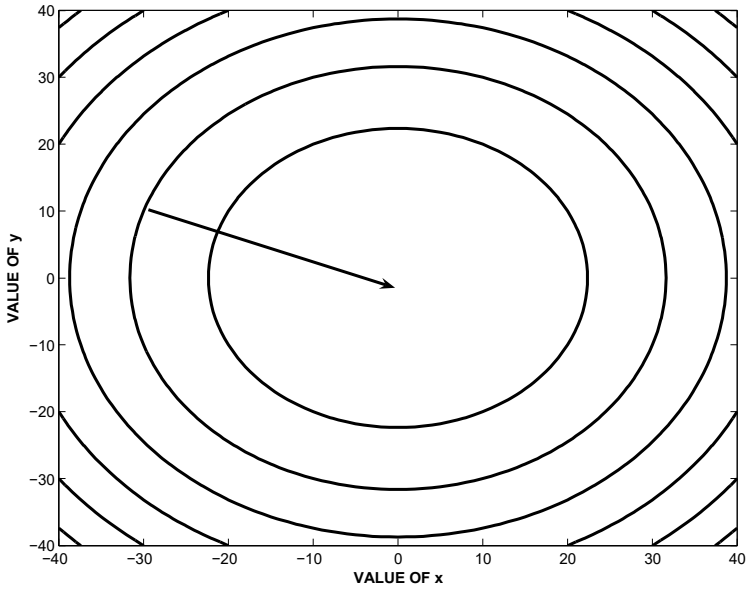
Neural Networks and Deep Learning, Springer, 2018
Chapter 3, Section 3.6

Revisiting the Vanishing and Exploding Gradient Problems

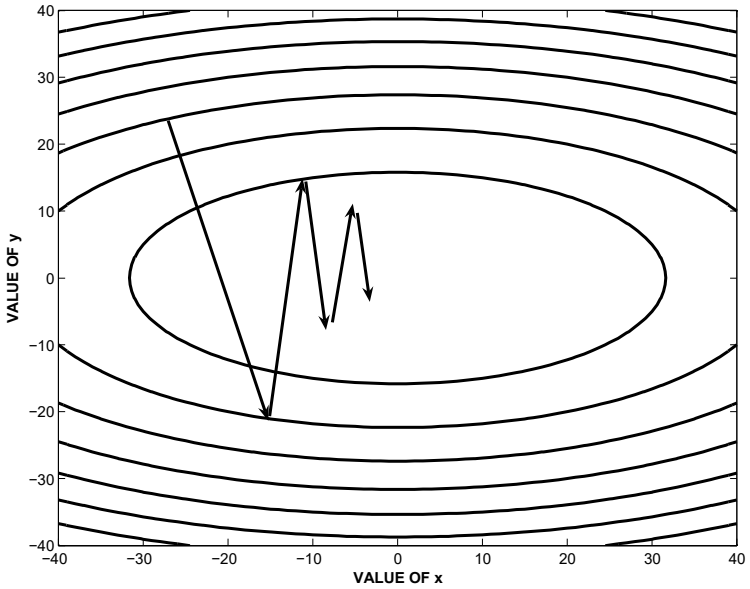


- Neural network with one node per layer.
- Forward propagation multiplicatively depends on each weight and activation function evaluation.
- Backpropagated partial derivative get multiplied by weights and activation function derivatives.
- Unless the values are exactly one, the partial derivatives will either continuously increase (explode) or decrease (vanish).
- Hard to initialize weights exactly right.

Revisiting the Bowl



(a) Loss function is circular bowl $L = x^2 + y^2$



(b) Loss function is elliptical bowl $L = x^2 + 4y^2$

- Varying scale of different parameters will cause bouncing

- Varying scale of features causes varying scale of parameters

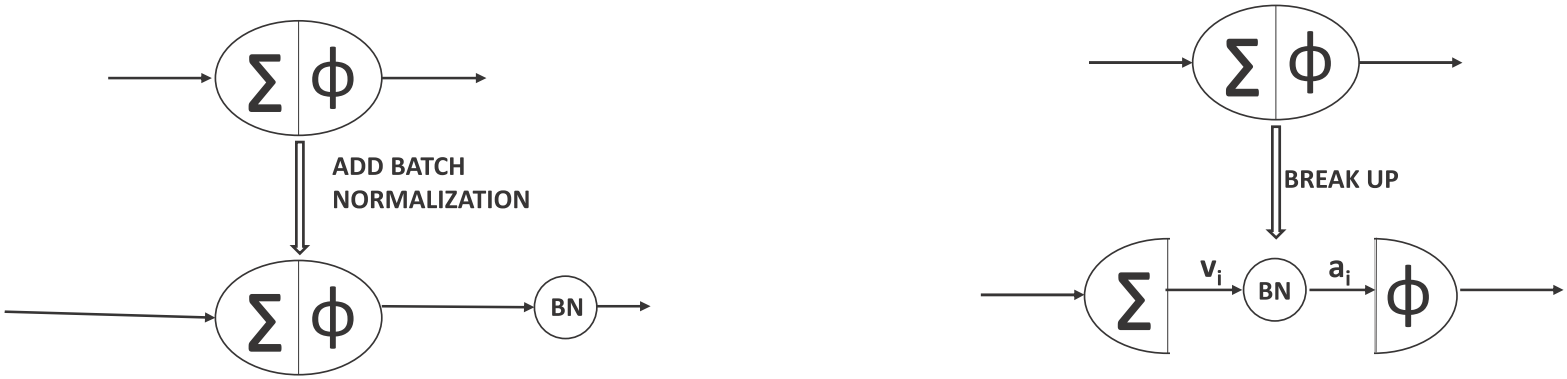
Input Shift

- One can view the input to each layer as a shifting data set of hidden activations during training.

- A shifting input causes problems during learning.
  - Convergence becomes slower.
  - Final result may not generalize well because of unstable inputs.

- Batch normalization ensures (somewhat) more stable inputs to each layer.

Solution: Batch Normalization



(a) Post-activation normalization (b) Pre-activation normalization

- Add an additional layer than normalizes in *batch-wise* fashion.
- Additional learnable parameters to ensure that optimal level of nonlinearity is used.
- Pre-activation normalization more common than post-activation normalization.

Batch Normalization Node

- The ith unit contains two parameters β_i and γ_i that need to be learned.

- Normalize over *batch* of m instances for ith unit.

$$\mu_i = \frac{\sum_{r=1}^{m} v_i^{(r)}}{m} \quad \forall i \qquad \text{[Batch Mean]}$$

$$\sigma_i^2 = \frac{\sum_{r=1}^{m} (v_i^{(r)} - \mu_i)^2}{m} + \epsilon \quad \forall i \quad \text{[Batch Variance]}$$

$$\widehat{v}_i^{(r)} = \frac{v_i^{(r)} - \mu_i}{\sigma_i} \quad \forall i, r \qquad \text{[Normalize Batch Instances]}$$

$$a_i^{(r)} = \gamma_i \cdot \widehat{v}_i^{(r)} + \beta_i \quad \forall i, r \quad \text{[Scale with Learnable Parameters]}$$

- Why do we need β_i and γ_i?

  - Most activations will be near zero (near-linear regime).

Changes to Backpropagation

- We need to backpropagate through the newly added layer of normalization nodes.
  - The BN node can be treated like any other node.
- We want to optimize the parameters β_i and γ_i.
  - The gradients with respect to these parameters are computed during backpropagation.
- Detailed derivations in book.

Issues in Inference

- The transformation parameters μ_i and σ_i depend on the batch.

- How should one compute them during testing when a *single* test instance is available?

- The values of μ_i and σ_i are computed up front using the *entire* population (of training data), and then treated as constants during testing time.

  - One can also maintain exponentially weighted averages during training.

- The normalization is a simple linear transformation during inference.

Batch Normalization as Regularizer

- Batch normalization also acts as a regularizer.
- Same data point can cause somewhat different updates depending on which batch it is included in.
- One can view this effect as a kind of noise added to the update process.
- Regularization is can be shown to be equivalent to adding a small amount of noise to the training data.
- The regularization is relatively mild.